

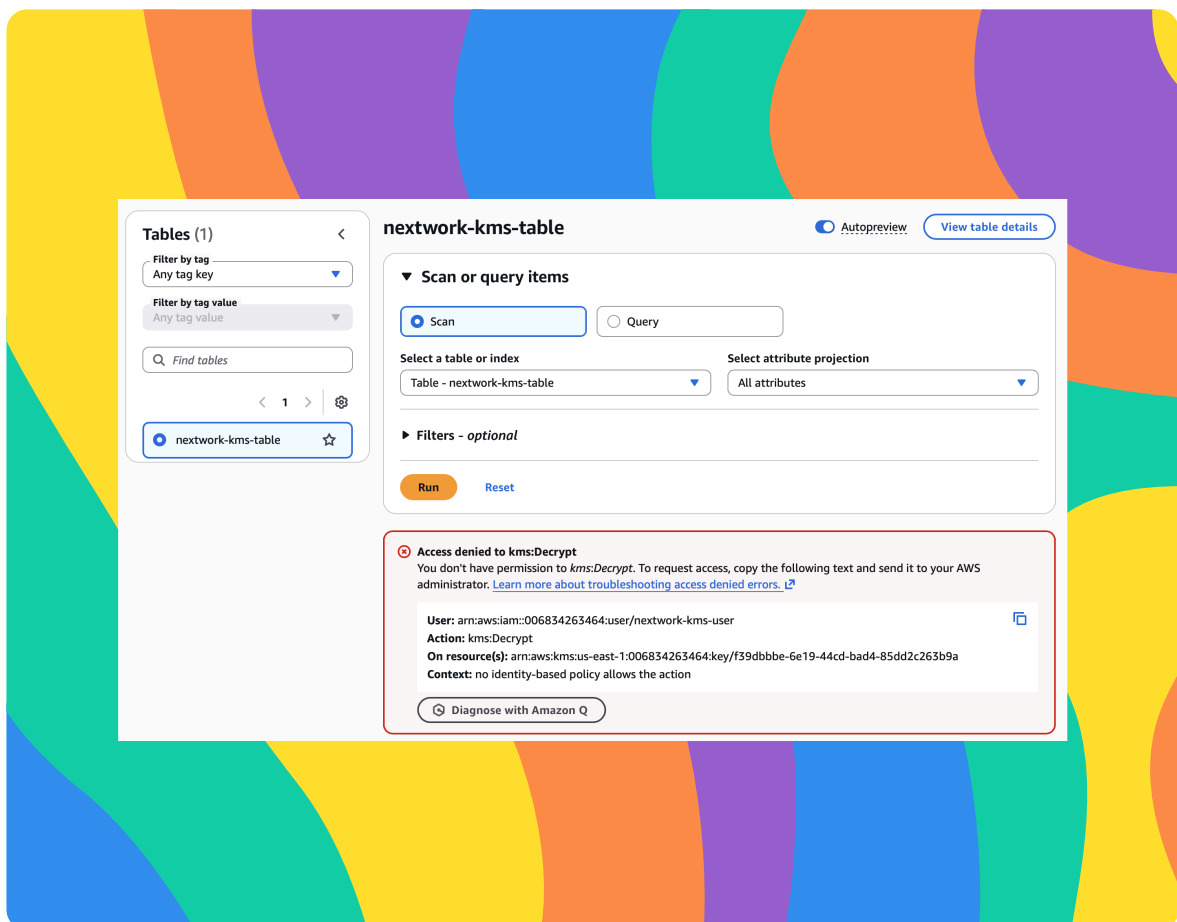


nextwork.org

Encrypt Data with AWS KMS

CH

Arpita Chowdhury





Introducing Today's Project!

In this project, I will demonstrate creating encryption keys with AWS KMS (Key Management Service), encrypt a DynamoDB database with a KMS key, ddd and retrieve data from database to test encryption, observe how AWS stops unauthorized access to data and give a user encryption access. The goal is to encrypt data with AWS KMS.

Tools and concepts

Services I used include AWS DynamoDB, AWS Key Management Service (KMS), and IAM for managing user permissions and enforcing secure access controls throughout the project. Key concepts I learned include encryption, key management, encryption at rest, IAM authorization, key policies, and how AWS evaluates permissions by combining IAM policies with KMS resource policies to determine whether a user can decrypt or encrypt data.

Project reflection

This project took me approximately 2 hours. The most challenging part was the secret mission It was most rewarding to complete the mission.



Arpita Chowdhury

NextWork Student

nextwork.org

I chose to do this project today because I want to learn about cloud security
Something that would make learning with NextWork even better is making
projects that reflect real life problems solved in a professional field and job.

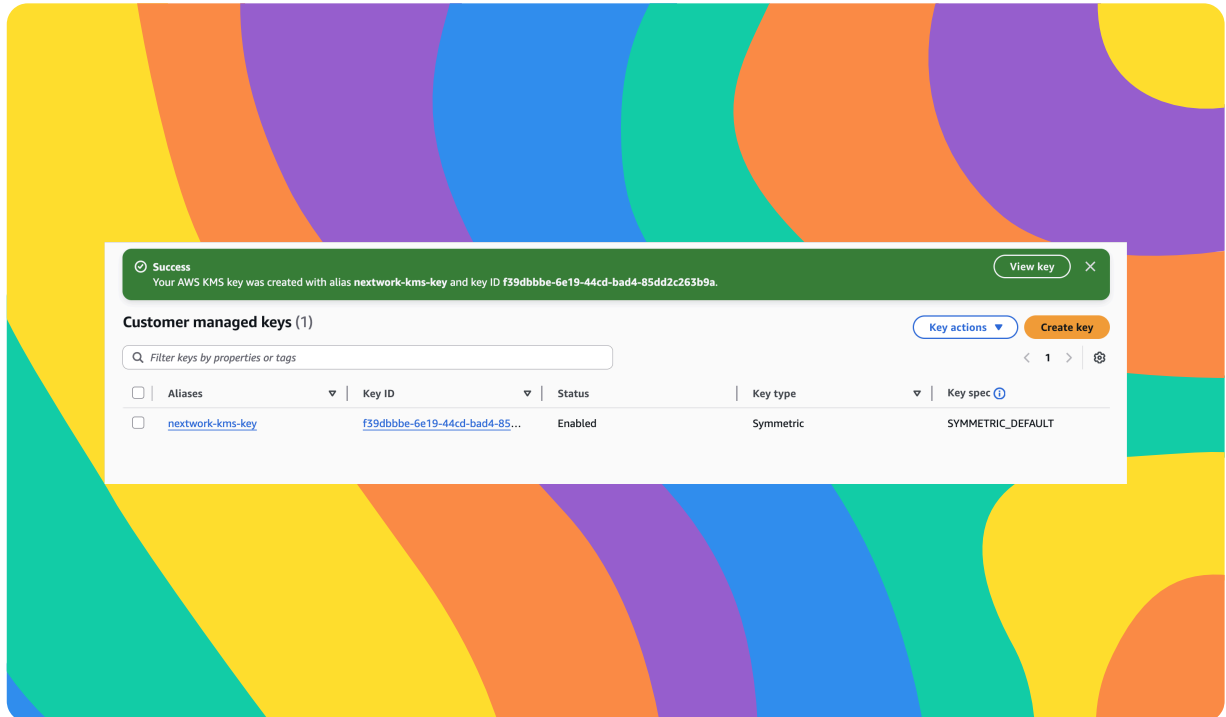


Encryption and KMS

Encryption is the process of converting readable data into an unreadable form using mathematical algorithms so that only authorized parties can access it; companies and developers use encryption to protect sensitive information such as personal data, financial records, passwords, and communications from unauthorized access or tampering, helping maintain privacy and comply with security standards; and encryption keys are the unique, often random pieces of information required to encrypt and decrypt the data, ensuring that only someone with the correct key can restore the original readable content.

AWS KMS is a managed Amazon Web Services service that lets organizations create, store, and control cryptographic keys used to encrypt their data, providing secure key generation, rotation, access control, and auditing without needing to manage hardware or complex infrastructure; key management systems are important because they centralize and protect encryption keys, ensure proper lifecycle management, reduce the risk of unauthorized access, support regulatory compliance, and help maintain consistent, secure encryption practices across applications and environments.

Encryption keys are broadly categorized as symmetric keys, where the same key is used to encrypt and decrypt data, and asymmetric keys, which use a public-private key pair for secure exchange and authentication; I set up a symmetric key because it offers faster performance, is efficient for encrypting large amounts of data, and is simpler to implement when both the sender and receiver can securely share the same secret key.

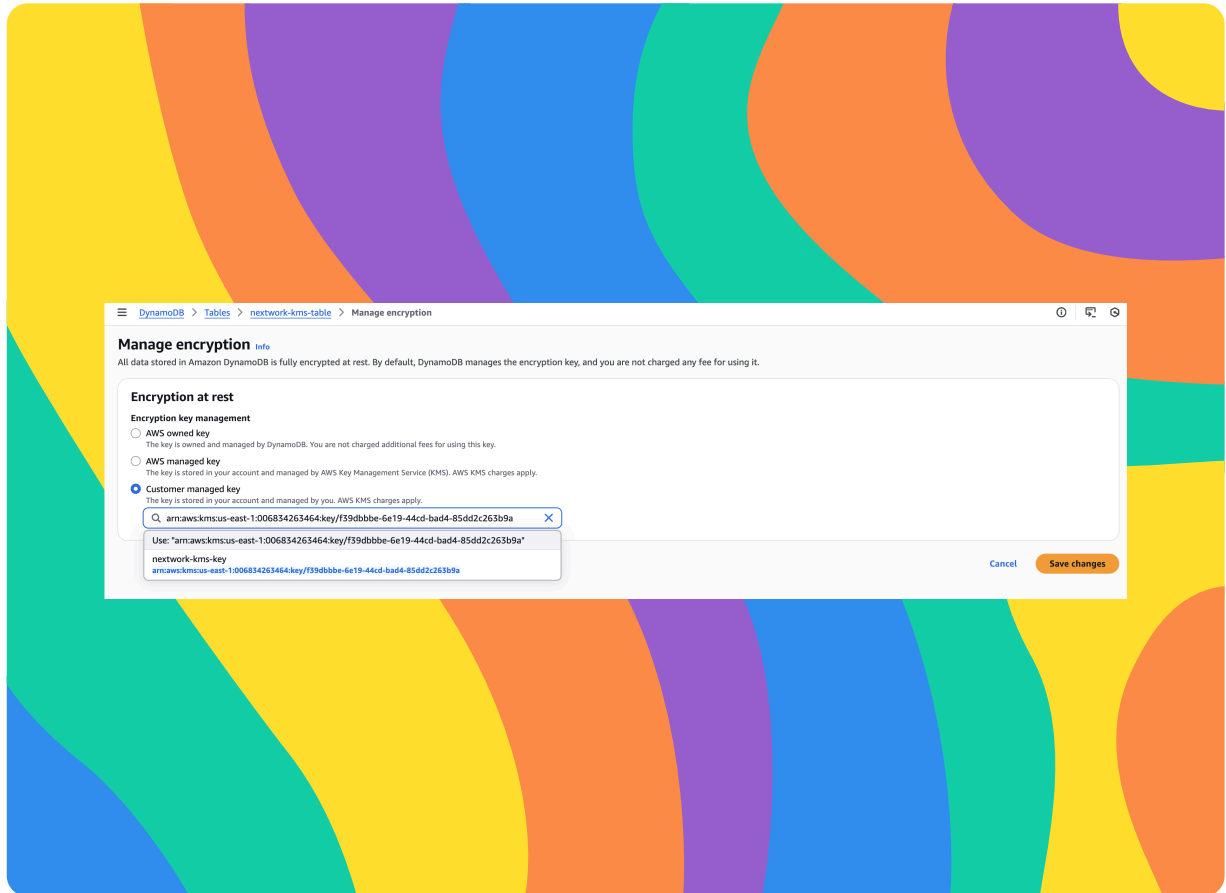




Encrypting Data

My encryption key will safeguard data in DynamoDB, which is a fully managed, serverless NoSQL database service designed to provide fast and predictable performance at any scale, and encryption ensures that any sensitive information stored in the table such as user records, application data, or system metadata remains protected at rest so that even if unauthorized individuals were to access the underlying storage, the data would remain unreadable without the proper decryption key.

The different encryption options in DynamoDB include AWS owned keys, AWS managed keys, and customer managed keys, and their differences are based on the level of control, customization, auditing, and responsibility you have over the encryption keys: AWS owned keys require no configuration and offer the least control, AWS managed keys provide basic control through AWS KMS with minimal setup, and customer managed keys give you full control over key policies, rotation, permissions, and monitoring; I selected a customer managed key because it allows me to explicitly control who can use the key, view detailed audit logs, manage its lifecycle, and ensure that my DynamoDB table's encryption aligns with stricter security and compliance requirements.





Data Visibility

ChatGPT said: Rather than controlling who has access to the key, KMS manages user permissions by evaluating IAM policies and key policies together to decide whether a user can perform a specific cryptographic action, such as `kms:Decrypt` or `kms:GenerateDataKey`. In other words, KMS does not simply grant access based on the user's identity it checks both the IAM permissions attached to the user and the resource-based policy on the KMS key itself, and access is only allowed when both policies explicitly permit the action.

ChatGPT said: Despite encrypting my DynamoDB table, I could still see the table's items because encryption at rest protects the underlying stored data, not the permissions of the user viewing it; DynamoDB uses transparent data encryption, which means that data is automatically encrypted when written to disk and automatically decrypted when accessed by an authorized user or application, so anyone with the proper IAM permissions to read the table will still be able to view the plaintext items regardless of whether the table is encrypted.



Arpita Chowdhury

NextWork Student

nextwork.org

Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2

Table: nextwork-kms-table - Items returned (1)

Scan started on December 11, 2025, 15:23:32

☐

id (String)

☐

[1](#)

< 1 >

⚙

Actions

Create item



Denying Access

I configured a new IAM user to grant the user full access to DynamoDB but not to my KMS key; the permission policies I granted this user are AmazonDynamoDBFullAccess but not kms:Decrypt, kms:GenerateDataKey, or any other permissions tied to my customer-managed KMS key, meaning the user can fully interact with DynamoDB resources but cannot use the encryption key required to read or write encrypted data.

After accessing the DynamoDB table as the test user, I encountered an “Access denied to kms:Decrypt” error because the user had full DynamoDB permissions but did not have the required KMS permissions to decrypt the table’s encryption key, which DynamoDB must use internally before it can return readable data; this confirmed that encryption at rest is enforced by KMS, and without explicit permissions such as kms:Decrypt and kms:GenerateDataKey on the customer-managed key, a user cannot read or query items from a DynamoDB table protected by that key even if they otherwise have full DynamoDB access.



Tables (1)

Filter by tag
Any tag key

Filter by tag value
Any tag value

Find tables

nextwork-kms-table

nextwork-kms-table Autopreview View table details

▼ Scan or query items

Scan Query

Select a table or index
Table - nextwork-kms-table

Select attribute projection
All attributes

Filters - optional

Run Reset

Access denied to kms:Decrypt
You don't have permission to `kms:Decrypt`. To request access, copy the following text and send it to your AWS administrator. [Learn more about troubleshooting access denied errors.](#)

User: arn:aws:iam::006834263464:user/nextwork-kms-user
Action: kms:Decrypt
On resource(s): arn:aws:kms:us-east-1:006834263464:key/f39dbbbe-6e19-44cd-bad4-85dd2c263b9a
Context: no identity-based policy allows the action

Diagnose with Amazon Q



EXTRA: Granting Access

To let my test user use the encryption key, I added the user's IAM ARN to the KMS key policy and granted the necessary cryptographic permissions, allowing the user to perform actions such as `kms:Decrypt`, `kms:Encrypt`, and `kms:GenerateDataKey`. My key's policy was updated to explicitly include the test user in the "Allow use of the key" statement, giving them permission to use the key while still maintaining control over who can administer or modify it, thereby enabling the user to successfully read and write encrypted data in DynamoDB.

Using the test user, I retried accessing the DynamoDB table and ran a scan again. I observed that the request succeeded and the table's items were now visible, which confirmed that once the user was granted the necessary KMS permissions through the key policy, DynamoDB was able to decrypt the data on their behalf and return readable results, demonstrating that access to encrypted data depends on both DynamoDB permissions and the appropriate KMS key permissions.

Encryption secures data instead of controlling who can access it, since it protects the data at rest but does not replace authorization mechanisms. I could combine encryption with IAM permission controls, fine-grained access policies, or DynamoDB item-level permissions to ensure that only approved users or applications can read or modify the data, providing both strong data protection and strict access management.



```
Resource :  
,  
{  
  "Sid": "Allow use of the key",  
  "Effect": "Allow",  
  "Principal": {  
    "AWS": [  
      "arn:aws:iam::006834263464:user/arpita-IAM-Admin",  
      "arn:aws:iam::006834263464:user/nextwork-kms-user"  
    ]  
  },  
  "Action": [  
    "kms:Encrypt",  
    "kms:Decrypt",  
    "kms:ReEncrypt*",  
    "kms:GenerateDataKey*",  
    "kms:DescribeKey"  
  ],  
  "Resource": "*"   
},
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

